ScreenAgent 与 UI-TARS 技术对比报告

一、引言

GUI Agent 的发展经历了三个阶段：基于规则的 Agent（Rule-based Agents）、Agent Framework（智能体框架）、Native Agent Model（原生智能体模型）。本报告聚焦于后两个阶段的代表性工作：

- ScreenAgent：属于 Agent Framework 阶段，采用显式的“规划-执行-反思”流水线，依赖专家设计的提示词和工作流。
- UI-TARS：属于 Native Agent Model 阶段，采用端到端模型，将感知、推理、动作、记忆统一在一个模型内。

此外，Claude Computer Use 与 UI-TARS 同属 Native Agent Model 阶段，采用纯视觉端到端架构。然而，由于 Claude Computer Use 的多模态模型训练过程是闭源的，其官方文档仅说明了开发者如何通过 API 使用该模型，并未披露模型架构、训练数据、训练方法等关键技术细节。因此，本报告仅对 UI-TARS 进行详细分析，未对 Claude Computer Use 展开同等深度的架构对比。

本报告分为两个部分：

- 第一部分（第二章）对比两者都具备的基本能力（感知、定位、执行、规划、反馈），分析其实现差异；
- 第二部分（第三章）对比无法简单归入基本能力的认知能力（记忆、快慢思考），并进一步提炼两者差异的核心概念。

第一部分回答“两者都能做什么、怎么做的”，第二部分回答“两者在认知层面有什么本质区别”。

二、基本能力对比

2.1 感知能力

维度	论文位置	核心思路
ScreenAgent	第 3.1 节 S-State Space	将屏幕截图作为环境的状态空间，收集动作执行前后的截图，分别表示每个动作前后的状态
UI-TARS	第 4.2 节 Enhancing GUI Perception	专门设计五类感知任务，显式训练模型的感知能力

两者都采用纯屏幕截图作为输入，不涉及 OCR 等文字识别库，也不需要访问 HTML 代码。但 UI-TARS 相比 ScreenAgent 的关键区别在于：UI-TARS 把感知能力划分为五个维度（Element Description、Dense Captioning、State Transition Captioning、QA、Set-of-Mark），并分别设计了训练任务来显式增强感知能力；而 ScreenAgent 没有专门的感知训练，感知能力是动作预测的副产品。

2.2 定位能力

模型	论文位置	核心思路
ScreenAgent	第 5.1 节 Evaluation Results on ScreenAgent Test-Set，Table 3	定位能力继承自 CogAgent 的视觉定位能力
UI-TARS	第 4.3 节 Improving Grounding Ability	构建“UI元素描述 → 坐标”的直接映射

ScreenAgent 的定位能力基于对已有模型 CogAgent 的微调。CogAgent 的原始定位精度仅为 0.07（判定标准：点击坐标是否落在标注的可行边界框内），通过视觉微调后提升至 0.51。UI-TARS 则为定位任务专门构建了数据集，采用自动爬取与人工标注相结合的方式，提取截图中元素的描述和边界框。同时，UI-TARS 对坐标进行了归一化处理（将坐标除以屏幕尺寸，转换为相对坐标），从而支持跨设备、跨分辨率的通用性。其定位训练任务为：输入一张屏幕截图 + 元素描述，输出归一化后的坐标。经过专门训练后，UI-TARS 实现了高精度定位，显著提升了点击和拖拽操作的精准度与有效性。

2.3 执行能力

模型	论文位置	核心思路
ScreenAgent	第 3.1 节 Computer Control Environment，Table 1	定义 JSON 格式的函数调用动作空间，通过 VNC 协议在桌面端执行
UI-TARS	第 4.3 节 Unified Action Modeling and Grounding，Table 1	设计统一动作空间，跨平台标准化，端到端输出归一化坐标

两者都通过鼠标和键盘操作来执行 GUI 动作，但动作空间的设计理念有显著差异。

ScreenAgent 的动作空间仅考虑桌面端，采用 JSON 格式的函数调用，可分为两大类：控制类动作包括 MouseAction、KeyboardAction 和 WaitAction；流程类动作包括 PlanAction（将复杂任

务拆解成一系列子任务）和 EvaluateSubTaskAction（评估子任务是否完成）。Agent 输出 JSON 后由环境解析并转换为 VNC 协议动作发送到被控电脑，鼠标位置相对于屏幕左上角。

UI-TARS 的动作空间则设计为跨平台统一动作空间，将动作分为共享动作（Click、Drag、Scroll、Type、Wait、Finished）和平台独有动作（桌面端：CallUser、Hotkey、LeftDouble；移动端：RightSingle、LongPress、PressBack、PressHome、PressEnter），以适配跨平台场景。其中 Finished() 和 CallUser() 是两个终端动作，分别用于任务完成和请求用户干预。在执行方式上，UI-TARS 直接输出归一化坐标（相对于屏幕尺寸），推理时端到端预测，无需外部 grounding 模块，从而支持跨设备、跨分辨率的通用性。

核心差异在于：其一，ScreenAgent 的动作空间包含流程类动作（PlanAction、EvaluateSubTaskAction），因为它是显式三阶段流水线，需要这些动作在规划、执行、反思三个阶段之间传递信息；而 UI-TARS 是端到端模型，规划和反思内化在模型推理中，因此动作空间只包含真正执行的控制类动作。其二，ScreenAgent 的动作空间面向单一桌面平台，坐标系统为屏幕绝对坐标；UI-TARS 的动作空间面向多平台统一设计，坐标系统为归一化相对坐标。其三，UI-TARS 通过统一动作空间实现跨平台知识迁移，而 ScreenAgent 不具备这一能力。

2.4 规划能力

模型	论文位置	核心思路
ScreenAgent	第 3.2 节 Control Pipeline，Planning Phase	显式的、独立的规划阶段，将复杂任务分解为子任务列表
UI-TARS	第 4.4 节 Infusing System-2 Reasoning	内化的、每步都有的推理，通过训练注入五种推理模式

两者都需要在感知之后、执行之前进行规划/推理，但实现方式有本质差异。

ScreenAgent 的规划是显式的、独立的阶段。在 Planning Phase 中，Agent 基于当前截图，依靠自身的常识知识和计算机知识，将复杂任务分解为子任务列表，输出 JSON 格式的 PlanAction。规划只在任务开始时执行一次（除非反思阶段判断需要重新规划）。附录 A 给出了完整的提示词模板，要求 Agent 输出类似如下的子任务列表：

```
{"action_type": "PlanAction", "element": "Open web browser"}
```

UI-TARS 的规划则是内化的、每步都有的推理。它在每个动作前生成 thoughts `t_i`，形成 `(o_1, t_1, a_1), (o_2, t_2, a_2), ...` 的交互序列。规划的注入通过三种方式实现：

1. 教程预训练：爬取约 600 万高质量 GUI 教程，为模型提供 GUI 知识和逻辑推理基础；
2. 思维标注：使用 ActRe 和 Thought Bootstrapping 为动作轨迹标注 thoughts，将数据从 `(o, a)` 扩展为 `(o, t, a)`；
3. 推理模式注入：引入五种推理模式——Task Decomposition（任务分解）、Long-term Consistency（长期一致性）、Milestone Recognition（里程碑识别）、Trial and Error（试错）、Reflection（反思）。

规划粒度为动作级，每个动作前都有推理。

2.5 反馈能力

模型	论文位置	核心思路
ScreenAgent	第 3.2 节 Control Pipeline，Reflecting Phase	显式的、独立的反思阶段，输出 EvaluateSubTaskAction 评估子任务状态
UI-TARS	第 4.5 节 Learning from Prior Experience in Long-term Memory	内化的反思能力，通过 Reflection Tuning 和 Agent DPO 训练注入

两者都需要在执行后评估动作结果、判断任务是否完成、决定是否重试或调整，但实现方式有本质差异。

ScreenAgent 的反馈是显式的、独立的阶段。在 Reflecting Phase 中，Agent 基于动作后的截图，评估当前子任务是否完成，输出 JSON 格式的 EvaluateSubTaskAction，包含三种状态：`sub_task_success`（子任务完成，继续下一个）、`need_retry`（需要重试，可给出 advice）、`need_reformulate`（需要重新制定整个计划）。反思在每次动作后执行，为流水线提供灵活性以处理不可预测情况。然而实验显示，所有模型在反馈阶段的准确率不足，GPT-4V 仅达到 0.60 F1，说明人类干预仍然必要。

UI-TARS 的反馈则是内化的、训练注入的能力。它不设独立的反思阶段，而是通过两种机制将反思能力内化到模型参数中：其一，Reflection Tuning（反思调优），包括 Error Correction（标注错误动作和纠正动作，形成正负样本对）和 Post-Reflection（模拟错误发生后的恢复步骤，训练模型补偿错误）；其二，Agent DPO（智能体直接偏好优化），利用正负样本对优化模型，增加纠正动作的概率，降低错误动作的概率。此外，UI-TARS 还通过 Online Trace Bootstrapping 在数百台虚拟机上自动收集、过滤、精炼交互轨迹，形成迭代训练闭环：$M_{n+1} = \text{FineTune}(M_n, \mathcal{T}_{\text{filtered},n})$。实验显示，DPO 显著提升了 OSWorld 性能（SFT 17.7 → DPO 18.7 / 22.7）。

核心差异：

- ScreenAgent 的反馈是显式的、独立的阶段，UI-TARS 的反馈是内化的、训练注入的能力。
- ScreenAgent 通过提示词引导重试或重新规划，UI-TARS 通过 DPO 训练让模型学会从错误中恢复。
- ScreenAgent 的反馈准确率低（GPT-4V 仅 0.60 F1），需要人类干预；UI-TARS 通过 Reflection Tuning 和 DPO 显著提升了错误恢复能力，最小化人类干预。
- ScreenAgent 的反馈不改变模型参数，UI-TARS 的反馈通过迭代训练持续更新模型参数，实现自我改进。

三、认知能力对比

第二章从功能层面（感知、定位、执行、规划、反馈）对比了两者的实现差异。但记忆和快慢思考无法简单归入上述任何一项基本能力——它们是跨阶段的认知能力，支撑所有阶段。本章先分别分析这两项认知能力（3.1、3.2），再进一步提炼两者差异的核心概念（3.3）。

3.1 记忆能力

UI-TARS 涉及记忆机制的论述分布在三个层面：

层面	论文章节	核心内容
理论层面	第 3.1 节	定义记忆的分类（短期记忆和长期记忆），并指出 Native Agent Model 与 Agent Framework 的本质区别——前者将长期记忆编码在模型参数中，后者需要外部记忆模块
架构层面	第 4.1 节	说明短期记忆的实现——保留最近 N 个观察，限制在 32k token 预算内，完整历史作为短期记忆保留
训练层面	第 4.5 节	说明长期记忆的形成——通过 Online Trace Bootstrapping 在数百台虚拟机上收集、过滤、精炼交互轨迹，通过迭代训练将经验编码到模型参数中，形成长期记忆

记忆分类

UI-TARS 将记忆分为两类：

类型	存储内容	实现方式	激活方式

短期记忆	任务特定信息，捕获 Agent 的即时上下文，包括动作历史、当前状态详情、任务执行轨迹	保留最近 N 个观察	直接作为模型输入
长期记忆	历史交互、任务和背景知识，包括先前任务的执行路径、用户偏好、任务操作经验	编码在模型内部参数中，将可观察的交互过程转化为隐式的、参数化的存储	In-Context Learning (ICL) 或 Chain-of-Thought (CoT) 推理

与 ScreenAgent 的对比

维度	**ScreenAgent**	**UI-TARS**
短期记忆	依赖 VLM 上下文窗口	保留最近 N 个观察，32k token 预算
长期记忆	无	编码在模型参数中
记忆模块	无	内化在模型参数中
跨任务学习	无	通过参数化存储实现
记忆激活	无	ICL / CoT 推理

3.2 快思考与慢思考

论文定位

UI-TARS 论文中，快慢思考的论述分布在三个层面：

层面	**论文章节**	**核心内容**
理论层面	第 3.1 节	定义 System 1（快思考）和 System 2（慢思考），并列出 System 2 的五个组成部分
训练层面	第 4.4 节	说明如何注入 System-2 推理，包括五种推理模式和 Thought Bootstrapping
实验层面	第 5.5 节	对比 System 1 和 System 2 在不同场景下的表现

Figure 3 和 Figure 6 提供了可视化说明。

双系统推理

UI-TARS 借鉴了人类认知的双过程理论（Groves & Thompson, 1970），将推理分为：

类型	特征	适用场景	优点	缺点
System 1（快思考）	快速、自动、直觉的思维	简单、常规任务（如点击熟悉的按钮）	响应迅速、效率高	局限于即时、反应性行为，无法处理多步任务
System 2（慢思考）	缓慢、审慎、分析的思维	复杂任务（如规划工作流、反思错误）	能处理复杂多步任务	单样本条件下可能引入错误推理步骤

System 2 的五种推理模式：

1. Task Decomposition（任务分解）
2. Long-term Consistency（长期一致性）
3. Milestone Recognition（里程碑识别）
4. Trial and Error（试错）
5. Reflection（反思）

与 ScreenAgent 的对比

维度	ScreenAgent	UI-TARS
快慢思考区分	无	有
推理模式	无显式注入	五种推理模式专门训练
规划粒度	子任务级	动作级（每步）

四、总结

ScreenAgent 是 Agent Framework 阶段的典型代表，它通过显式的“规划-执行-反思”流水线和专家设计的提示词来组织工作流，感知和定位能力是动作微调的副产品；UI-TARS 是 Native Agent Model 阶段的代表，它将感知、推理、动作、记忆统一在端到端模型中，通过大规模数据专门训练感知和定位能力，并具备记忆和快慢思考等高级认知能力。